Daily Scholar Papers Report — 2026-05-26

2026-05-26

Daily Scholar Papers Report — 2026-05-26

Download PDF

Window covered: 2026-05-25 → 2026-05-26 (Google Scholar alerts + user-curated self-emails, last 24 h)

Executive Summary

Today's window surfaced four Scholar-alert threads carrying nine distinct candidates plus one user-curated self-email; seven cleared Stage-1 triage (six on followed-researcher signal plus the preference-aware preprint posterior of 0.75, and one user-pick that bypasses saturation/topicality filters). The three Outstanding deep-reads converge on a single architectural move — *use the LLM as a semantic proposal engine, and discharge every soundness-relevant verdict on a deterministic, reproducible backend*: **"SoK: DARPA's AI Cyber Challenge (AIxCC)"** (Zhang et al., Georgia Tech / TAMU / Microsoft / Kudu Dynamics / SIFT) is the first systematic study of all seven AFC finalist Cyber Reasoning Systems built on the complete competition database plus discussions with every team, distilling the de-facto bug-candidate → PoV-generation → fuzz-reproduce → patch pipeline and showing fuzzer-driven and LLM-driven PoV pipelines converge as complementary, mutually-seeding loops across 63 challenge CPVs; **"Constraint-Driven Fuzzing at Scale with FANDANGO"** (Zamudio Amaya, Smytzek, Liggesmeyer, Huber, Zeller — CISPA, ICST '26 tool track) replaces ISLa's SMT solving with a genetic-search loop over grammar + Python semantic constraints, achieving 29× to 1,355× throughput speed-up across CSV/reST/ScriptSizeC/TAR/XML at >99 % validity, packaged with EUPL v1.2 release, classroom-tested at scale, and already used as the AUTOSPEC backend in industry; **"SymTEE"** (Ma, Shi, Han, Liu, Niu, Lo — SMU, FORGE '26) lets an LLM (GPT-5) synthesise KLEE-compatible harnesses with lightweight stubs from AST-sliced TEE code, reaching 100 % precision and 92.3 % recall on 26 vulnerabilities at ~$0.05 per analysis with no real TEE setup. Four Keep papers extend orthogonal axes — **TEERepair** (companion to SymTEE) hits 95.45 %/91.01 % P/R for issue locating with a DSL-guided LLM patcher; **FuzzingBrain V2** (TAMU) tops the AIxCC C/C++ final at 90 % detection and surfaces 29 zero-day vulns / 2 CVEs in the wild; **QuartetFuzz** (TAMU + Aisle) defines harness correctness as P1–P4 source-level conditions and exposes a 25-year-latent OpenSSL DES stack-buffer-overread by auditing 586 OSS-Fuzz harnesses; **KnitFuzz** (Syracuse, CODASPY '26) tackles socket-syscall fuzzing's C1–C4 (abstract wrappers / weak typing / implicit dependencies / remote-endpoint influence) and finds two kernel bugs, with up to +29.84 pp branch coverage over Syzkaller / Moonshine.

Outstanding: 3 · **Keep:** 4 · **Borderline High-Priority:** 0

The full analysis follows.

Highlighted Papers

#	Title	Authors	Venue	Link
1.1	SoK: DARPA's AI Cyber Challenge (AIxCC): Competition Design, Architectures, and Lessons Learned	Cen Zhang, Younggi Park, Fabian Fleischer, Yu-Fu Fu, Jiho Kim, Dongkwan Kim, Youngjoon Kim, Qingxiao Xu, Andrew Chin, Ze Sheng, Hanqing Zhao, Brian J. Lee, Joshua Wang, Michael Pelican, David J. Musliner, Jeff Huang, Jon Silliman, Mikel Mcdaniel, Jefferson Casavant, Isaac Goldthwaite, Nicholas Vidovich, Matthew Lehman, Taesoo Kim	arXiv preprint, 2026	arXiv:2602.07666
1.2	Constraint-Driven Fuzzing at Scale with FANDANGO	José Antonio Zamudio Amaya, Marius Smytzek, Alexander Liggesmeyer, Valentin Huber, Andreas Zeller	*ICST '26* tool track (CISPA preprint)	CISPA 32346339
1.3	Finding Missing Input Validation in TEEs via LLM-Assisted Symbolic Execution	Chengyan Ma, Jieke Shi, Ruidong Han, Ye Liu, Yuqing Niu, David Lo	*ACM FORGE '26* (CC-BY 4.0)	arXiv:2605.22058
2.1	Automated Repair of TEE Partitioning Issues via DSL-Guided and LLM-Assisted Patching	Chengyan Ma, Jieke Shi, Ruidong Han, Ye Liu, Feng Li, Yuqing Niu, David Lo	arXiv preprint, 2026 (CC-BY 4.0)	arXiv:2605.22087
2.2	FuzzingBrain V2: A Multi-Agent LLM System for Automated Vulnerability Discovery and Reproduction	Ze Sheng, Zhicheng Chen, Qingxiao Xu, Kewen Zhu, Jeff Huang	arXiv preprint, 2026	arXiv:2605.21779
2.3	Quality-Assured Fuzz Harness Generation via the Four Principles Framework	Ze Sheng, Dmitrijs Trizna, Luigino Camastra, Zhicheng Chen, Qingxiao Xu, Jeff Huang	arXiv preprint, 2026	arXiv:2605.21824
2.4	KnitFuzz: LLM-guided Kernel Fuzzing via Context-Sensitive Socket System Calls	Siwei Zhang, Endadul Hoque	*ACM CODASPY '26* (CC-BY 4.0)	DOI 10.1145/3800506.3803511

Outstanding Deep-Reads

1.1 · AGENTIC-CRS-SOK · [USER-PICK] The first systematic study of all seven AFC-finalist Cyber Reasoning Systems built on the complete AIxCC competition database, finalist whitepapers / source code, and discussions with every team — across 63 CPVs (33 C, 30 Java), distils a de-facto bug-candidate → PoV → fuzz-reproduce → patch pipeline and shows fuzzer-driven and LLM-driven PoV pipelines converge as mutually-seeding loops

1.1 SoK: DARPA's AI Cyber Challenge (AIxCC): Competition Design, Architectures, and Lessons Learned

arXiv:2602.07666

Title: SoK: DARPA's AI Cyber Challenge (AIxCC): Competition Design, Architectures, and Lessons Learned **Authors:** Cen Zhang, Younggi Park, Fabian Fleischer, Yu-Fu Fu, Jiho Kim, Dongkwan Kim, Youngjoon Kim, Qingxiao Xu, Andrew Chin, Ze Sheng, Hanqing Zhao, Brian J. Lee, Joshua Wang, Michael Pelican, David J. Musliner, Jeff Huang, Jon Silliman, Mikel Mcdaniel, Jefferson Casavant, Isaac Goldthwaite, Nicholas Vidovich, Matthew Lehman, Taesoo Kim **Affiliations:** Georgia Institute of Technology · Texas A&M University · Smart Information Flow Technologies (SIFT) · Kudu Dynamics · Independent Researcher · Microsoft **Venue:** arXiv preprint, v2 2026-02-18. **License:** arXiv non-exclusive distribution. **Source signal:** STEP 2b user-pick (forwarded via self-email 2026-05-25 10:44 UTC); also preference-promoted preprint (`venue::preprint` posterior 0.75).

Thesis

DARPA's AI Cyber Challenge (AIxCC, 2023–2025) is the largest competition to date for building fully autonomous Cyber Reasoning Systems (CRSs) that leverage recent advances in AI — particularly large language models — to discover and remediate vulnerabilities in real-world open-source software. Yet despite the scale of the event, no systematic account had crystallised across the seven finalist teams: each whitepaper described one CRS's choices, organisers reported aggregate scoreboards, and the architectural patterns that actually drove competition success or failure remained scattered. This paper closes that gap with a primary-source study that examined all seven finalist CRS codebases, the complete competition database (challenges, results, execution traces), and structured discussions with both organisers and every finalist team — distilling the design vocabulary the field needs to design the next competition and to deploy autonomous CRSs in practice.

Architecture

The systematisation organises around three research questions, each backed by primary-source evidence:

1. **RQ1 — How is AIxCC designed to guide and evaluate AI-driven vulnerability discovery and patching?** Walks through the AFC dataset (63 Challenge Project Vulnerabilities — 33 C, 30 Java, drawn from critical-infrastructure OSS), the dispatch / Competition-API submission infrastructure, the per-CPV scoring rubric, and the design decisions that defined what a "valid" PoV and "valid" patch even mean. Identifies SARIF validation and bundle submission as competition-design choices that materially shape what CRSs build.

2. **RQ2 — What architectural and technical approaches did finalist teams employ?** Per-team architectural distillation — *RoboDuck (Team TI)* organises the entire system around "bug candidates" with agentic patch-generation feedback loops, simplifying architecture by letting bug candidates be the central data type; *Artiphishell (Team SP)* maximises technical-coverage breadth, implementing the widest variety of techniques across fuzzing, static, and dynamic analysis; *BugBuster (Team 42)* takes a pragmatic technology-choice approach — simple and stable over comprehensive; traditional fuzzing for bug-finding with LLM-driven patch synthesis. Two complementary PoV-generation pipelines emerge across teams: **fuzzer-driven** (improve fuzzing coverage and seed quality with LLM-supplied inputs; then mine harness crashes for bugs) and **LLM-driven** (LLMs identify and filter bug candidates first, then synthesise PoVs targeting those candidates). Crucially, LLM-generated PoV attempts — *even unsuccessful ones* — serve as seeds boosting the fuzzer's effectiveness, closing a feedback loop between the two pipelines.
3. **RQ3 — What insights emerge from competition results beyond the final scoreboard?** Re-analyses per-CPV outcomes against foundational baselines, exposing where the apparent scoreboard ranking diverges from real technical advances; identifies which design choices in §5 (bug-candidate centric vs technical-coverage centric vs pragmatic) actually translated into per-CPV success, and which categories of bugs remained outside the reach of every system. Java vs C patterns differ: Java's logical vulnerabilities often stem from API misuse patterns LLMs handle well, while C's memory-safety bugs continue to require fuzzer-reproducible witnesses.

Headline numbers

Aspect	Number / claim
AFC CPV dataset	**63 Challenge Project Vulnerabilities** (33 C, 30 Java) drawn from critical-infrastructure OSS
Finalist CRSs studied	**All 7** AFC finalists — primary sources: codebases + whitepapers + organiser data + team discussions
Patch pipeline	**De-facto 4-stage** (localise candidates → agent-orchestrated patch synthesis → functional + security validation → feedback-driven refinement) emerges across every CRS
PoV pipelines	**Two complementary** — fuzzer-driven and LLM-driven, mutually seeding each other

Why it matters

This is the only systematic ground-truth account of how seven independently-engineered CRSs converged (and diverged) on the same architectural building blocks under DARPA's evaluation harness. The cross-team RQ3 distillation defines the next 12–18 months of agentic-CRS research direction — *which architectural choice mattered at the CPV level, which only mattered for scoreboard ranking, which produced zero real lift*. The patch-pipeline taxonomy (localise → synth → validate → refine) and the fuzzer ↔ LLM mutual-seeding insight are immediately reusable framings for any LLM-+-fuzzing work; the methodology — *gather primary sources from every finalist immediately after the event* — is a model template for "competition-driven SoK" that future autonomous-security competitions can borrow as-is.

1.2 · GENETIC-FUZZ · A constraint-driven fuzzer that replaces ISLa's SMT solving with a genetic-search loop over grammar + Python semantic constraints — 29× to 1,355× throughput speed-up across CSV / reST / ScriptSizeC / TAR / XML at >99 % validity, EUPL v1.2 release with 353-page docs, classroom-tested, and already the AUTOSPEC backend behind LLM-synthesised RFC specifications at Volkswagen

1.2 Constraint-Driven Fuzzing at Scale with FANDANGO

CISPA 32346339 · Tool site & docs · Screencast

Title: Constraint-Driven Fuzzing at Scale with FANDANGO **Authors:** José Antonio Zamudio Amaya, Marius Smytzek, Alexander Liggesmeyer, Valentin Huber, Andreas Zeller **Affiliations:** CISPA Helmholtz Center for Information Security, Germany **Venue:** ICST '26 tool track (CISPA preprint, figshare DOI 32346339). **License:** Tool released under **EUPL v1.2**; paper is CISPA preprint copy. **Source signal:** Scholar alert "Andreas Zeller - new articles" 2026-05-25, plus preference-promoted preprint.

Thesis

ISLa achieved 100 % validity on a benchmark of complex input languages, but at the lowest throughput of any tool in that comparison — its SMT-based constraint solver is the bottleneck, and many real input formats simply cannot be fuzzed at scale through Z3. FANDANGO's claim is that for grammar-+-constraint fuzzing the constraint solver is the binding constraint, not the model; *replace symbolic solving with a genetic-search loop over the grammar*, let semantic constraints be expressed in plain Python rather than a limited SMT-LIB function-solver dialect, and the throughput ceiling shifts by one to three orders of magnitude without sacrificing validity.

Architecture

The evolutionary loop (paper §II, Fig 4) decomposes into eight numbered stages:

1. **Parse specification** — read a single-file grammar plus Python constraints.
2. **Initialise population** — random grammar expansion produces the initial inputs.
3. **Evaluate** — each input scored against constraints, producing a fitness in [0, 1] (1 = fully satisfied, 0 = violated, partial values reflecting *proximity to the target* — the crucial design choice that lets evolution gradient-descend toward valid inputs).
4. **Selection** — high-scoring individuals become parents.
5. **Crossover** — identifies the failing sub-trees responsible for constraint violations and swaps structurally compatible sub-trees between parents, so offspring remain grammatically valid by construction.
6. **Mutation** — syntactically valid mutations maintain diversity.
7. **Solution** — the loop terminates when all constraints are satisfied; the best inputs are emitted.
8. **Program under test** — generated test suite is fed downstream.

The constraint language exposes five primitives (paper §II): *value-shaping constraints* (`where 3 <= len(str(<str>)) <= 10`, `where int(<int>) != 113`); *quantifier constraints* over collected node iterables (`*<char>` collects all `<char>` as a Python iterable, then `any(...)` / `all(...)` give existential / universal forms, `not(any(...))` for negated existentials); *optimization goals* separate from boolean constraints, guiding the search toward smaller or larger values; *generator functions* in Python encoding semantic dependencies (e.g. `hashlib`-derived hash fields) that a grammar alone cannot express; and per-rule *probabilistic* weights to bias the initial population.

Headline results — five subjects, one-hour budget each

Subject	Approach	inputs/s	Validity (%)	Speedup
CSV 1.0	ISLa	2.06	100.00	—
	FANDANGO	**2,792.39**	99.74	**1,355×**
reST 0.21.2	ISLa	8.18	100.00	—
	FANDANGO	**1,195.34**	99.27	**146×**
ScriptSizeC 0.9.28rc	ISLa	6.85	100.00	—
	FANDANGO	**200.75**	99.51	**29×**
TAR 3.5.3	ISLa	0.28	100.00	—
	FANDANGO	**13.33**	100.00	**48×**
XML 1.3.0	ISLa	2.38	100.0	—
	FANDANGO	**433.70**	99.67	**183×**

Why it matters

Two-fold significance. On *evaluation*: the one-to-three-orders-of-magnitude speed-up across every benchmark while sacrificing at most 0.73 pp of validity is a decisive comparative result — the *replace-SMT-with-genetic-search* design choice is the right answer to ISLa's known throughput bottleneck. On *deployment*: 353 pages of documentation, EUPL v1.2 packaging with CLI + Python APIs, hundreds of students at university courses and summer schools moving from tutorial grammars to their own formats inside a single class session, and the AUTOSPEC project at CISPA (industry collaboration stemming from Volkswagen) already using FANDANGO as the downstream test generator behind LLM-synthesised RFC protocol specifications. The combination — strong empirical numbers plus a tool ready for the next "spec-synth → fuzz" pipeline — places this paper at the centre of the year's grammar-fuzzing conversation. Methodology highly reusable for any team that has grammars plus semantic constraints but couldn't afford ISLa's solver cost.

1.3 · LLM-SYMEX-TEE · An LLM (GPT-5) synthesises KLEE-compatible harnesses with lightweight stubs directly from AST-sliced TEE code — 100 % precision and 92.3 % recall on 26 vulnerabilities at ~$0.05 / analysis, with no real TEE setup required, accepted at FORGE '26

1.3 Finding Missing Input Validation in TEEs via LLM-Assisted Symbolic Execution

arXiv:2605.22058 · DOI 10.1145/3793655.3793740 · Paper PDF

Title: Finding Missing Input Validation in TEEs via LLM-Assisted Symbolic Execution **Authors:** Chengyan Ma, Jieke Shi, Ruidong Han, Ye Liu, Yuqing Niu, David Lo **Affiliations:** Singapore Management University **Venue:** ACM FORGE '26 — 2026 IEEE/ACM Third International Conference on AI Foundation Models and Software Engineering, April 12–13 2026, Rio de Janeiro. **License: Creative Commons Attribution 4.0 International (CC-BY 4.0)** — paper p. 1 footnote. **Source signal:** Scholar alert "David Lo - 新文章" 2026-05-25, plus preference-promoted preprint.

Thesis

Trusted Execution Environments (TEEs) provide hardware-enforced isolation that protects sensitive code and data, but analysing TEE applications for missing input validation remains punishingly hard: setting up a complete TEE build and runtime environment (Intel SGX, ARM TrustZone, vendor-specific toolchains, binary signing) is costly and complex, and hardware isolation makes standard debugging methods ineffective because crashes or coverage data cannot be observed across the secure-world boundary. SymTEE's claim is that with the right *LLM-+-symbolic-execution* division of labour, none of that hardware setup is needed: an LLM converts statically-sliced TEE code into a self-contained KLEE harness with mock stubs, and KLEE — running locally, no TEE involved — discharges the missing-validation verdict.

Architecture

Three-stage pipeline (paper §3, Figure 4):

1. **AST-based slicing** locates TEE code regions that may lack sufficient input validation. SymTEE walks the AST to find the field specifying the size of memory operations whose data originates from the untrusted normal world (e.g. the `dkLen` parameter passed across the TEE boundary into `g_CryptoTaPbkdf_PBKDF2`). The slice is the smallest self-contained code region that exercises the suspected missing check.
2. **LLM harness synthesis (GPT-5 in the paper)** expands each slice into a complete KLEE program with minimal stubs. Stubs are deliberately *side-effect-free* — `void TEE_MemMove(...) {}` is annotated `/* Stubbed: do nothing to avoid actual memory side effects */` — so KLEE focuses entirely on the path conditions that distinguish a missing-validation execution from a safe one. The harness embeds `#include <klee/klee.h>` and exposes the input via `klee_make_symbolic`.
3. **KLEE symbolic execution** runs locally and systematically explores feasible paths, reporting any path that reaches an unsafe memory operation without an intervening size check. SymTEE then synthesises a fix suggestion (e.g. `if (dkLen > 512) return TEE_ERROR_BAD_PARAMETERS;` inserted before the unsafe `TEE_MemMove(output, resultBuf, dkLen)`).

The walkthrough example (paper Figure 2) shows SymTEE detecting a missing input-length validation in the `shuaifengyun/basicAlg_use` PBKDF2 implementation on GitHub — an attacker-controlled `dkLen` could trigger a buffer overflow on the fixed-size output buffer, and SymTEE's suggested fix gates the copy on a bound check before any TEE memory is touched.

Headline numbers — Table 1

Corpus	Detection precision	Detection recall	False negatives	Token cost
11 real-world + 15 synthetic (PartitioningE-Bench + real projects)	**100 %**	**92.3 %**	2	**~$0.05 / vuln** (~5,931 tokens)

Why it matters

Three converging properties. *Decisive numbers* — 100 % precision and 92.3 % recall at two orders of magnitude lower per-bug cost than prior TEE-fuzzing campaigns. *Architectural cleanliness* — the *LLM-generates-the-symbolic-execution-harness-with-stubs* pattern decouples LLM creativity from any soundness-relevant verdict; KLEE alone determines whether a path is feasible. *Generalisability* — the same shape applies to any platform whose runtime is too heavy to instrument (TEEs, hypervisors, baseband, kernel modules, BMC firmware): replace the hardware environment with LLM-synthesised stubs that the symbolic executor can reason about locally. Together with §2.1 (TEERepair, the same group), this forms an end-to-end detect-then-repair pipeline for TEE partitioning issues.

Keep — Concise Cards

2.1 · LLM-REPAIR-TEE · DSL-guided LLM patcher for TEE partitioning issues — 95.45 % / 91.01 % P/R on issue locating (F1 0.93), patches confirmed and merged upstream across multiple TEE projects, CC-BY 4.0

2.1 Automated Repair of TEE Partitioning Issues via DSL-Guided and LLM-Assisted Patching (TEERepair)

arXiv:2605.22087 · Paper PDF

Authors: Chengyan Ma, Jieke Shi, Ruidong Han, Ye Liu, Feng Li, Yuqing Niu, David Lo (Singapore Management University) **Venue:** arXiv preprint, 21 May 2026 (companion to §1.3 SymTEE). **License: CC-BY 4.0** (paper p. 1 footnote).

Method. Five-stage pipeline: (1) static partitioning-issue detector reuses §1.3-style slice extraction; (2) DSL of structured *repair templates* — developers / debuggers express transformations as templates the patcher can parameterise; (3) LLM-assisted patch generator instantiates templates from issue context; (4) validator re-runs the detector to confirm the issue is gone and feeds errors back to the LLM; (5) optional functional-correctness check via automated test execution where TEE projects ship client programs (many do not — explicit limitation noted).

Headline numbers (Table 3). On PartitioningE-Bench, TEERepair achieves **95.45 % precision / 91.01 % recall / F1 0.93** for issue locating, substantially better than the bare LLM baseline (91.53 % / 60.67 % / 0.73). 89 validated vulnerable cases used for repair-stage evaluation; patches confirmed and merged by maintainers across multiple TEE projects (e.g. `basicAlg_use` — 5,162 LoC).

Why keep. Closes the loop with SymTEE (detect → repair) for the full TEE-partitioning workflow; the DSL-template + re-detector validation pattern is the right level of abstraction to constrain LLM patching to semantically meaningful transformations rather than free-form text edits.

2.2 · AGENTIC-VULN-DISC · Multi-agent LLM system topping AIxCC 2025 Final C/C++ at 90 % detection (36/40), with 29 zero-day vulnerabilities and 2 CVEs in real-world OSS deployment

2.2 FuzzingBrain V2: A Multi-Agent LLM System for Automated Vulnerability Discovery and Reproduction

arXiv:2605.21779

Authors: Ze Sheng, Zhicheng Chen, Qingxiao Xu, Kewen Zhu, Jeff Huang (Texas A&M University) **Venue:** arXiv preprint, 20 May 2026. **License:** arXiv non-exclusive.

Method. Four contributions: (1) fully automated vulnerability analysis built on Google's OSS-Fuzz, ensuring every reported vulnerability is fuzzer-reproducible — eliminates sanitizer-noise false positives by definition; (2) *Suspicious Point* — a novel control-flow-based abstraction for vulnerability localization at an intermediate granularity between line-level (insufficient context) and function-level (context too extensive); (3) logic-driven hierarchical function analysis with dual-layer fuzzing enhancing function coverage under resource constraints; (4) MCP-based static + dynamic analysis tools with context engineering for reasoning about vulnerabilities with complex cross-function dependencies and triggering conditions.

Headline numbers. AIxCC 2025 Final Competition C/C++ dataset — **90 % detection rate (36 of 40 vulnerabilities), ranking first among all systems**. Real-world deployment — **29 zero-day vulnerabilities** across **12 OSS projects**, all confirmed and fixed by maintainers, **2 CVE IDs assigned**.

Why keep. Strong empirical numbers from a public competition with substantial real-world impact. The *Suspicious Point* abstraction is a useful intermediate-granularity primitive worth borrowing in any LLM-driven localization pipeline.

2.3 · HARNESS-QA · P1–P4 source-level definition of harness correctness + adversarial-probe gate — audited 586 OSS-Fuzz harnesses, surfaced 53 quality violations and a 25-year-latent OpenSSL DES stack-buffer-overread, 4.8 % FP rate in deployment

2.3 Quality-Assured Fuzz Harness Generation via the Four Principles Framework (QuartetFuzz)

arXiv:2605.21824

Authors: Ze Sheng, Dmitrijs Trizna, Luigino Camastra, Zhicheng Chen, Qingxiao Xu, Jeff Huang (Texas A&M / Aisle) **Venue:** arXiv preprint, 20 May 2026 (ACM template, conference target). **License:** ACM-template default copyright.

Method. Defines the *Four Principles* of harness correctness as source-level conditions: **P1 Logic Correctness** (free of stale state, resource leaks, bad data handling); **P2 API Protocol Compliance** (valid call order, object lifecycles, parameter constraints); **P3 Security Boundary Respect** (exercise public interfaces, not internal-only code); **P4 Entry Point Adequacy** (target attack-surface-relevant entries, not helpers). Operationalised by *Adversarial Probing* (P1/P2 runtime sub-checks) and static call-graph reachability (P3/P4 boundary sub-checks). Four-stage pipeline = entry-selection agent ranks by static danger score → API-research agent collects call order & lifecycle info → static-driven bounded build loop → adversarial-validation gate forces every selected probe to pass before submission. Organises fuzzing around *Logic Groups* (functional units like "parse a PNG image" or "complete a TLS handshake") rather than isolated functions.

Headline numbers. Audited **586 production OSS-Fuzz harnesses** across 70 C/C++ projects → **53 quality violations identified, 45 confirmed by maintainers, 35 fixed / merged upstream**; the audit exposed **2 latent library bugs** the harnesses had been masking, including an OpenSSL DES stack-buffer-overread latent over **25 years**. On the 100-harness gold-standard dataset, QuartetFuzz matches human-written harnesses on coverage (TOST ±2pp equivalence, $p<10^{-10}$ on both line and branch) and outperforms OSS-Fuzz-Gen by 6.9–8.3 pp and PromeFuzz by 4.1–5.2 pp. Deployment on 23 OSS

projects (C/C++/Java/JavaScript) → **42 bug reports, 29 fixed/confirmed, 3 CVEs, 4.8 % FP rate**; P1/P2 checks blocked 58 harness-induced false-positive crashes upstream.

Why keep. Principled definition of harness correctness + auditor-then-generator deployment story makes this the new reference point for harness-QA. The maintainer-trust framing (58 % of contacted OSS-Fuzz maintainers never responded; 10 % declined outright) makes a useful empirical case for *trust over raw coverage* as the right success metric.

2.4 · LLM-KERNEL-FUZZ · LLM-guided socket-syscall kernel fuzzer with a dynamic remote-endpoint peer program — +19.38 pp basic-block coverage / +29.84 pp branch coverage over Syzkaller / Moonshine, two kernel bugs (one previously unknown), CODASPY '26 CC-BY 4.0

2.4 KnitFuzz: LLM-guided Kernel Fuzzing via Context-Sensitive Socket System Calls

DOI 10.1145/3800506.3803511 · Paper PDF · Author copy

Authors: Siwei Zhang, Endadul Hoque (Syracuse University) **Venue:** ACM CODASPY '26 — Conference on Data and Application Security and Privacy, 23–25 June 2026, Frankfurt am Main.

Method. Identifies four challenges for socket-syscall kernel fuzzing: **C1 Abstract Wrappers** (e.g. `listen()` dispatches differently for TCP vs UDP); **C2 Weak Typing** (e.g. `setsockopt()`'s `void* optval` interpretation depends on the first three args — `level = SOL_SOCKET` routes to `sock_setsockopt()`, `level = SOL_TCP` to `tcp_setsockopt()`); **C3 Implicit Dependencies** (e.g. enabling `SO_REUSEADDR` via `setsockopt()` silently changes `bind()` behaviour — Syzlang's resource constraints cannot express this); **C4 Remote Endpoint Influence** (a TCP `send()` requires a real `connect()` response — without a remote endpoint, fuzzing stalls). Two-stage decoupled architecture: dataset generator samples 5–9 syscalls per combination → prompt generator queries LLM iteratively until T valid C programs are produced → verifiers (static analyzer + code checker + compiler) gate every candidate → refinement loop until iteration limit → ELF binaries fed to AFL++ in a VM with KCOV → *runtime manager* dynamically spawns a remote-endpoint peer program when `isPeerNeeded(D)`, providing bidirectional network exchanges. *Selective memory instrumentation* mutates only the instrumented memory locations rather than all coverage-relevant ones. Program generation and fuzzing run independently so generation does not throttle fuzzing throughput.

Headline numbers. Multiple Linux kernels — up to **+19.13–19.38 pp basic-block coverage** and **+14.12–29.84 pp branch coverage** over Syzkaller and Moonshine. Two kernel bugs discovered, including a previously unknown vulnerability (both unpatched at writing). Empirical pain-point on Syzkaller documented from Syzbot: **61.5 % reproduction success rate**; >70 % of bug reports ultimately ignored or discarded by the Linux kernel community because they cannot be reproduced reliably — motivating the *deterministic-verifier-gates-every-seed-program* design.

Formal Algorithm 1 sketch. `while |P| ≠ T: funcs ← S[index++]; P ← GenerateAndVerify(G, {syscalls: funcs}); if P ≠ ε: P ← GenerateAndVerify(M, {syscalls, program: P}); if P ≠ ε: if isPeerNeeded(D): φ ← GenerateAndVerify(P, {program: P}); if φ ≠ ε: add (P, φ); else: add (P, ε); return P`.

Why keep. The dynamic remote-endpoint design choice is the missing piece in every Syzkaller-derived networking fuzzer. C1–C4 gives any kernel-fuzzing follow-on a ready-made gap analysis to cite; the *candidate-then-verify-then-fuzz* scaffolding is the same pattern as §1.3, §2.1, §2.3 — independent groups converging on a stable design discipline.

Cross-Paper Synthesis

A consistent architectural pattern runs through every Outstanding / Keep paper in today's batch — *LLM proposes, deterministic backend disposes*:

- **SymTEE** (§1.3): LLM proposes the KLEE harness with stubs, KLEE disposes the verdict.
- **TEERepair** (§2.1): LLM proposes the patch, DSL template plus re-running the static detector disposes whether the issue is actually fixed.
- **FANDANGO** (§1.2): the spec language proposes constraints, a fitness-driven evolutionary loop disposes which inputs satisfy them — replacing SMT solving with genetic search at 29× to 1,355× throughput, without sacrificing validity.
- **QuartetFuzz** (§2.3): LLM proposes the harness, an *adversarial probe* discharges P1–P4 before any crash is submitted.
- **FuzzingBrain V2** (§2.2): LLM identifies *Suspicious Points*, OSS-Fuzz reproducibility check discharges every vulnerability claim.
- **KnitFuzz** (§2.4): LLM generates context-sensitive C programs, static analyzer + compiler verifiers + AFL++ coverage gate discharge them before fuzzing; the dynamic remote endpoint peer closes the C4 loop.
- **AIxCC SoK** (§1.1, user-pick): empirical confirmation that all seven independently-engineered AFC finalist CRSs converged on the *bug-candidate identification → PoV-generation → fuzz-reproduce → patch* shape; both fuzzer-driven and LLM-driven PoV pipelines feed each other in a hybrid loop.

Two follow-on observations.

Validation gates have crystallised into a design vocabulary. P1–P4 (QuartetFuzz), `isPeerNeeded(D)` peer-program verification (KnitFuzz), KLEE-stub symbolic execution (SymTEE), DSL repair templates with re-detector validation (TEERepair), and `GenerateAndVerify` refinement loops (KnitFuzz Algorithm 1) are all instances of the same pattern — *every LLM output that could change a soundness-relevant verdict is forced through a deterministic check*. A near-term meta-paper could systematise these gates the way last week's SecDev SoK systematised symbolic-execution modules.

Throughput vs validity is the binding constraint, not LLM quality. FANDANGO's ISLa-vs-genetic-search comparison shows that for grammar-+-constraint fuzzing the SMT solver, not the model, is the bottleneck — replacing it with evolutionary search yields 29× to 1,355× speed-up at >99 % validity. Expect more "deterministic-engine replacement" papers (e.g. KLEE / Z3 alternatives) over the next year.

Writing & Rationale Insights

The FORGE-CC-BY footnote signal: ACM's FORGE proceedings format places a CC-BY 4.0 footnote on page 1 of every accepted paper, so SymTEE and TEERepair are both CC-BY 4.0 by virtue of this default — a useful default-by-venue pattern to remember when assessing embedding rights. CODASPY's CC-BY 4.0 default does the same for KnitFuzz.

EUPL v1.2 for tool papers: FANDANGO licenses itself under the European Union Public License v1.2 — the European answer to MIT/Apache, copyleft-compatible. Useful precedent for any tool-track paper out of CISPA / Inria / TU Munich groups.

The “AIxCC SoK” trick: Zhang et al. validate a powerful methodological move — *don’t wait until DARPA publishes, ask each finalist team directly for source + design docs + execution traces and write the SoK while the event is fresh*. The fact that all seven teams cooperated says something about the field’s current openness; the paper is a model template for “competition-driven SoK.” Worth borrowing the structure for any future systematisation of competitive ML-security work.

Headline-number framing discipline: every Outstanding paper today leads with a compact (precision / recall / speedup / coverage Δ) headline drawn straight from the abstract; following that habit in our own write-ups consistently produces more readable cards. The chart-vs-narrative ratio matters less than the *single decisive number per paper* habit.